

- For an introduction to threads, see [https://www.cs.uic.edu/~jbell/CourseNotes/OperatingSystems/4\\_Threads.html](https://www.cs.uic.edu/~jbell/CourseNotes/OperatingSystems/4_Threads.html).
- The full pthread API docs are available online; just search for “pthread documentation.”
- For documentation on the Windows thread API, search for “Process and Thread Functions” on <https://msdn.microsoft.com/>.
- For more information on thread scheduling, search for Nikita Ishkov’s “A Complete Guide to Linux Process Scheduling” online.
- For a great introduction to Go’s implementation of coroutines (which are known as “goroutines”), watch this presentation by Rob Pike: <https://www.youtube.com/watch?v=f6kdp27TYZs>.

## 4.5 Introduction to Concurrent Programming

There’s a wide range of explicitly parallel computing hardware out there, but how can we take advantage of it as programmers? The answer lies in *concurrent programming* techniques. In concurrent software, a workload is broken down into two or more flows of control that can run semi-independently. As we saw in Section 4.1, in order for a system to qualify as concurrent, it must involve multiple readers and/or multiple writers of *shared* data.

Rob Pike, a Distinguished Engineer at Google Inc. who specializes in distributed and concurrent systems and programming languages, defines concurrency as “the composition of independently executing computations.” This definition underscores the idea that the multiple flows of control in a concurrent system normally operate semi-independently, but their computations are *composed by sharing data* and by *synchronizing* their operations in various ways.

Concurrency can take many forms. Some examples include:

- a piped chain of commands running under Linux or Windows, such as `cat render.cpp | grep "light"`,
- a single process comprised of multiple threads that share a virtual memory space and operate on a common dataset,
- a thread group comprised of thousands of threads running on a GPU, all cooperating to render a scene,
- a multiplayer video game, sharing a common game state between clients running on multiple PCs or game consoles.

### 4.5.1 Why Write Concurrent Software?

Concurrent programs are sometimes written because a model of multiple semi-independent flows of control simply matches the problem better than a single flow-of-control design. A concurrent design might also be chosen to best make use of a multicore computing platform, even if the problem at hand might be more naturally suited to a sequential design.

### 4.5.2 Concurrent Programming Models

In order for the various threads within a concurrent program to cooperate, they need to *share data*, and they need to *synchronize* their activities. In other words, they need to *communicate*. There are two basic ways in which concurrent threads can communicate:

- *Message passing.* In this communication mode, concurrent threads pass *messages* between one another in order to share data and synchronize their activities. The messages might be sent across a network, passed between processes using a pipe, or transmitted via a message queue in memory that is accessible to both sender and receiver. This approach works both for threads running on a single computer (either within a single process or across multiple processes), and for threads within processes running on physically distinct computers (e.g., a computer cluster or a grid of machines spread across the globe).
- *Shared memory.* In this communication mode, two or more threads are granted access to the same block of physical memory, and can therefore operate directly on any data objects residing in that memory area. Direct access to shared memory only works when all threads are running on a single computer with a bank of physical RAM that can be “seen” by all CPU cores. Threads within a single process always share a virtual address space, so they can share memory “for free.” Threads within different processes can also share memory by mapping certain physical memory pages into all of the processes’ virtual address spaces.

It’s interesting to note that the *illusion* of shared memory between physically separate computers can be implemented on top of a message-passing system—this technique is known as *distributed shared memory*. Likewise, a message-passing mechanism can be implemented on top of a shared memory architecture, by implementing a message queue that resides in the shared memory pool.

Each approach has its pros and cons. Physically-shared memory is the most efficient way to share a large amount of data, because that data doesn't have to be copied for transmission between threads. On the other hand, as we'll see in Sections 4.5.3 and 4.7, the *sharing* of resources of any kind (memory or other resources) brings with it a host of synchronization problems that tend to be difficult to reason about, and are very tricky to account for in a manner that guarantees correctness of the program. A message-passing design tends to lessen the impacts of (but not eliminate) these kinds of problems.

In this book, we'll focus primarily on shared memory concurrency. We do this for two reasons: First, this is the kind of concurrency you're most likely to encounter as a game programmer, because game engines are typically implemented as single-process multithreaded programs. (Networked multiplayer games are a notable exception to this rule, since they make heavy use of message passing.) Second, shared memory concurrency is a more difficult topic to get your head around. Once you understand concurrency within a shared memory environment, message passing techniques should prove relatively easy to learn.

### 4.5.3 Race Conditions

A *race condition* is defined as any situation in which the behavior of a program is dependent on timing. In other words, in the presence of a race condition, the behavior of the program can *change* when the relative sequence of events occurring across the system changes, due to variability in the lengths of time taken by the various flows of control to perform their tasks.

#### 4.5.3.1 Critical Races

Sometimes race conditions are innocuous—the behavior of the program may change somewhat depending on timing, but the race causes no ill effects. On the other hand, a *critical race* is a race condition that has the potential to cause *incorrect* program behavior.

The kinds of bugs caused by critical races often seem “strange” or even “impossible” to programmers who aren't experienced with them. Examples include:

- intermittent or seemingly random bugs or crashes,
- incorrect results,
- data structures that get into corrupted states,
- bugs that magically disappear when you switch to a debug build,

- bugs that are around for a while, and then go away for a few days, only to return again (usually the night before E3!),
- bugs that go away when logging (a.k.a., “`printf()` debugging”) is added to the program in an attempt to discover the source of the problem.

Programmers often call these kinds of issues Heisenbugs.

#### 4.5.3.2 Data Races

A *data race* is a critical race condition in which two or more flows of control interfere with one another while reading and/or writing a block of shared data, resulting in data corruption. Data races are *the* central problem of concurrent programming. Writing concurrent programs always boils down to eliminating data races, either by carefully controlling access to shared data, or by replacing shared data with private, independent copies of data (thereby transforming a concurrency problem into a sequential programming problem).

To better understand data races, consider the following simple snippet of C/C++ code:

```
int g_count = 0;

inline void IncrementCount()
{
    ++g_count;
}
```

If you compiled this code for an Intel x86 CPU and viewed the disassembly, it would look something like this:

```
mov    eax,[g_count]    ; read g_count into register EAX
inc    eax              ; increment the value
mov    [g_count],eax    ; write EAX back into g_count
```

This is an example of a *read-modify-write* (RMW) operation.

Now imagine that two threads A and B were both to call the `IncrementCount()` function concurrently (either in parallel or via preemptive multithreading). Under normal operation, if each thread called the function exactly once, we’d expect the final value of `g_count` to be 2, because either thread A increments `g_count` and then thread B increments it, or vice-versa. This is illustrated in Table 4.1.

Next let’s consider the case of our two threads running on a single-core machine with preemptive multitasking. Let’s say that thread A runs first, and



| Thread A  |     | Thread B  |     | Value of<br>g_count |
|-----------|-----|-----------|-----|---------------------|
| Action    | EAX | Action    | EAX |                     |
|           | ?   |           | ?   | 0                   |
| Read      | 0   |           | ?   | 0                   |
| Increment | 1   |           | ?   | 0                   |
| Write     | 1   |           | ?   | 1                   |
|           | 1   | Read      | 1   | 1                   |
|           | 1   | Increment | 2   | 1                   |
|           | 1   | Write     | 2   | 2                   |

Table 4.1. Example of correct operation of a simple two-threaded concurrent program. First thread A reads the contents of a shared variable, increments the value, and writes the results back into the shared variable. At a later time, thread B does the same steps. The final value of the shared variable is 2 as expected.

| Thread A  |     | Thread B  |     | Value of<br>g_count |
|-----------|-----|-----------|-----|---------------------|
| Action    | EAX | Action    | EAX |                     |
|           | ?   |           | ?   | 0                   |
| Read      | 0   |           | ?   | 0                   |
|           | 0   | Read      | 0   | 0                   |
|           | 0   | Increment | 1   | 0                   |
|           | 0   | Write     | 1   | 1                   |
| Increment | 1   |           | 1   | 1                   |
| Write     | 1   |           | 1   | 1                   |

Table 4.2. Example of a race condition. Thread A reads the value of the shared variable, but then is preempted by thread B, which also reads the (same) value. By the time both threads have incremented the value and written their results back to the shared variable, the global variable contains the incorrect value 1 instead of the expected value of 2.

has just finished executing the first `mov` instruction when a context switch to thread B occurs. Instead of thread A executing its `inc` instruction, thread B runs its first `mov` instruction. After some time thread B's quantum expires, and the kernel context switches back to thread A, which continues where it left off and executes the `inc` instruction. Table 4.2 illustrates what happens. Hint: It's not good! The final value of `g_count` is no longer 2 as it should be.

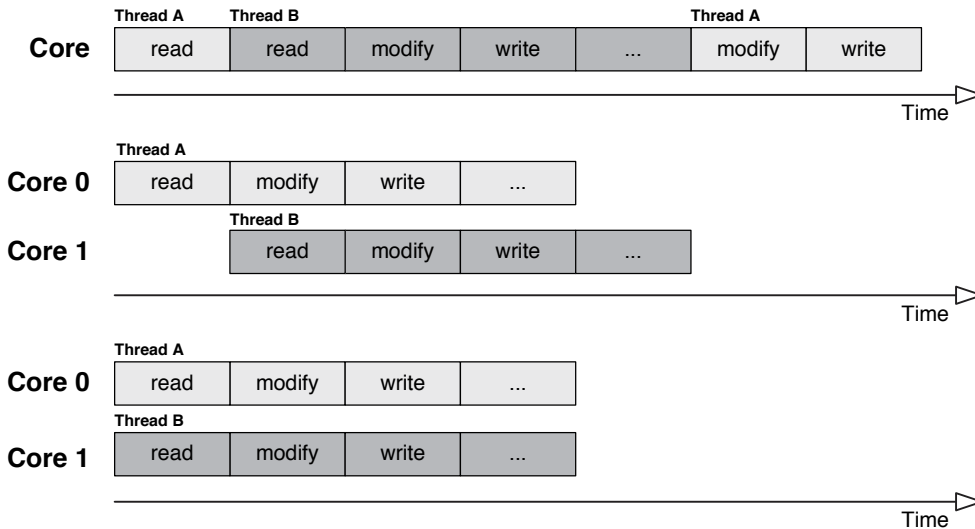


Figure 4.27. Three ways in which a data race can occur within a read-modify-write operation. Top: Two threads racing on a single CPU core. Middle: Two threads overlapping on two separate cores and offset by one instruction. Bottom: Two threads overlapping in perfect synchronization on two cores.

If we run our two threads on parallel hardware, a similar bug can occur, although for a slightly different reason. As in the single-core case, we might get lucky: The two read-modify-write operations might not overlap at all, and the result will be correct. However, if the two read-modify-write operations overlap, either offset from one another or in perfect synchronization, both threads can end up loading the same value of `g_count` into their respective `EAX` registers. Both will increment the value, and both will write it to memory. One thread will overwrite the results of the other, but it doesn't really matter—because they both loaded the same initial value, the final value of `g_count` will end up being incorrect, just as it was in the single-core scenario. The three data race scenarios (preemption, offset overlap, and perfect synchronization) are illustrated in Figure 4.27.

#### 4.5.4 Critical Operations and Atomicity

Whenever one operation is interrupted by another, we have the potential for a data race bug. However, not all interruptions actually cause bugs. For example, if a thread is performing an operation on a chunk of data that can only be “seen” by that one thread, no data race can occur. Such an operation can be interrupted at any moment by any other operation without consequence.

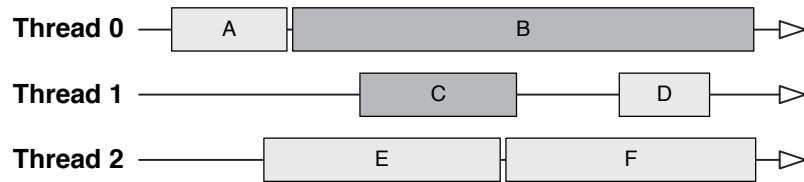


Figure 4.28. Because each step in an algorithm takes a finite amount of time to be performed, it becomes difficult to answer questions about the relative ordering of the steps in a multithreaded program. For example, does operation B happen before or after operation C?

Likewise, if an operation on one data object is interrupted by an operation on a different object, there's no way for those two operations to interfere with one another;<sup>6</sup> and hence no data race bugs are possible. Data race bugs only occur when an operation on a shared object is interrupted by another operation on that *same* object. So we can only talk meaningfully about data races in relation to a particular shared data object.

Let's use the term *critical operation* to refer to any operation that can possibly read or mutate one particular shared object. To guarantee that the shared object is free from data race bugs, we must ensure that none of its critical operations can interrupt one another. When a critical operation is made uninterruptible in this manner, it is called an *atomic operation*. Alternatively, we can say that such an operation has the property of *atomicity*.

#### 4.5.4.1 Invocation and Response

When we first learn to program, we're usually taught that the *time* required to perform each step in an algorithm is not relevant to the correctness of the algorithm—all that matters is that the steps are performed in the proper *order*. This simple model works well for sequential (single-threaded) programs. But in the presence of multiple threads, it's not possible to define the order of a set of operations when those operations each have a finite duration. This idea is illustrated in Figure 4.28.

In a concurrent system, the only way to define the notion of *order* is to restrict ourselves to talking about *instantaneous events*. Given any pair of instantaneous events, there are only three possibilities: event A happens before event B, event A happens after event B, or the two events are simultaneous. (Perfectly simultaneous events are rare, but they can occur in a multicore computer in which some or all cores share a synchronized clock.)

Any operation with a finite duration can be broken down into two instantaneous events—its *invocation* (the moment at which the operation begins) and

<sup>6</sup>This is only strictly true if the two objects reside on different cache lines.

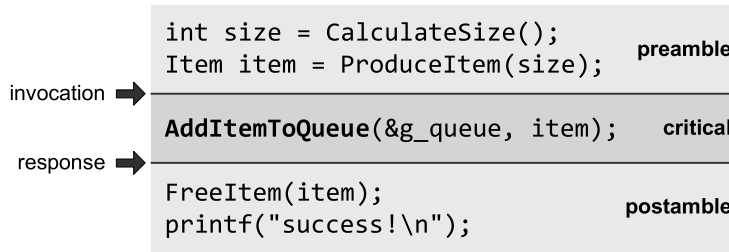


Figure 4.29. Any code snippet that includes a critical operation can be partitioned into three sections. The critical operation itself is bounded above by its invocation, and below by its response.

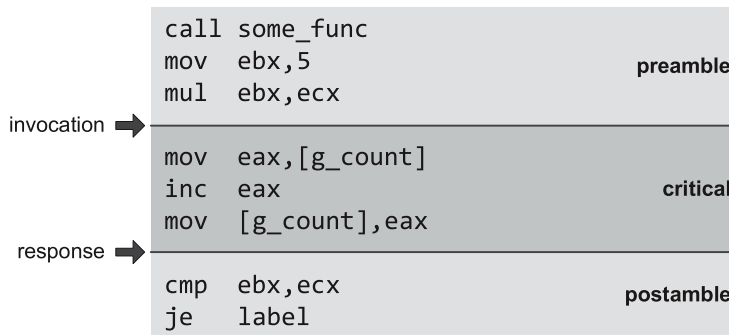


Figure 4.30. Another example, this time in assembly language, of a code snippet partitioned into three sections, bounded by the invocation and response of a critical operation.

its *response* (the moment at which it is considered to be complete). When we look at any code snippet that includes a critical operation on some shared data object, we can thus divide it into three sections, with the instantaneous invocation and response events demarking the boundaries between them. Note that we're talking here about the order in which events occur *as they were written* in the source code—this is known as *program order*.

- *Preamble section:* All code that occurs before the critical operation's invocation, in program order.
- *Critical section:* The code that comprises the critical operation itself.
- *Postamble section:* All code that occurs after the critical operation's response, in program order.

This notion of partitioning a block of code into three sections is illustrated in Figures 4.29 and 4.30.

#### 4.5.4.2 Atomicity Defined

As we saw in Section 4.5.3.2, a data race bug can occur when a critical operation is interrupted by another critical operation on the same shared object. This can happen:

- when one thread preempts another on a single core, or
- when two or more critical operations overlap across multiple cores.

Thinking in terms of invocation and response, we can pin down the general notion of interruption a bit more precisely: An *interruption* occurs whenever the invocation and/or response of one operation occurs *between* the invocation and response of another operation. But as we've said, not all interruptions lead to data race bugs. A critical operation on a particular shared object can only be affected by a data race if its invocation and response are interrupted by another critical operation on that *same* object. Therefore, we can define the *atomicity* of a critical operation as follows:

A critical operation can be said to have executed *atomically* if its invocation and response are not interrupted by another critical operation on that same object.

We should stress here that it's perfectly fine for a critical operation to be interrupted by other noncritical operations, or by critical operations affecting other unrelated data objects. Only when two critical operations on the *same* object interrupt one another does a data race bug occur. Figure 4.31 illustrates various cases—one in which a critical operation succeeds in executing atomically, and three in which it does not.

We can guarantee that a critical operation will be executed atomically if we can make it appear, from the point of view of all other threads in the system, to have occurred *instantaneously*. In other words, it must appear as though the invocation and response of the operation are simultaneous, or that the critical operation itself has a zero duration. That way, there can be no possibility of another critical operation's invocation or response "sneaking in" between the invocation and response of the operation in question.

#### 4.5.4.3 Making an Operation Atomic

But how can we transform a critical operation into an atomic operation? The easiest and most reliable way to accomplish this is to use a special object called a *mutex*. A mutex is an object provided by the operating system that acts like a padlock, in that it can be locked and unlocked by a thread. Given two critical

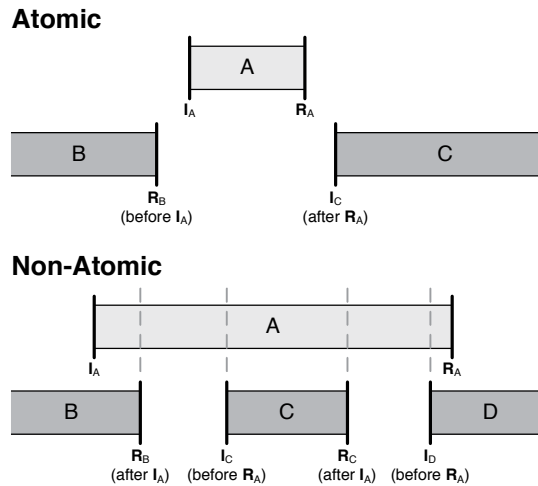


Figure 4.31. Top: Critical operation A can be said to have executed atomically, because it was not interrupted by any other invocations or responses from critical operations on the same shared object. Bottom: Three scenarios in which critical operation A executed nonatomically because it was interrupted by the invocation and/or response of another critical operation on the same object.

operations on a particular shared data object, we guard the invocation of each operation with the acquisition of the mutex, and release the mutex at each one's response. Because the OS guarantees that a mutex can only be acquired by one thread at a time, we can thus be certain that the invocation or response of one operation can never happen in between the invocation and response of the other. From the point of view of the global ordering of the events in a concurrent system, a critical operation guarded with a mutex lock *appears* to be instantaneous.

Mutexes are part of a collection of concurrency tools provided by the operating system known as *thread synchronization primitives*. We'll explore thread synchronization primitives in Section 4.6.

#### 4.5.4.4 Atomicity as Serialization

Consider a group of threads, all attempting to perform a single operation on a shared data object. Without atomicity, these operations might happen simultaneously, or they might overlap in all sorts of unpredictable ways over time. This is illustrated in Figure 4.32.

However, making the operation atomic guarantees that only one thread will ever be performing it at any given moment in time. This has the effect of *serializing* the operations—what used to be a jumble of overlapping opera-

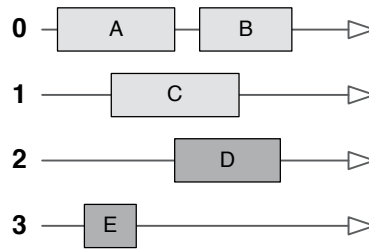


Figure 4.32. Without atomicity, operations performed by multiple threads can overlap in unpredictable ways over time.

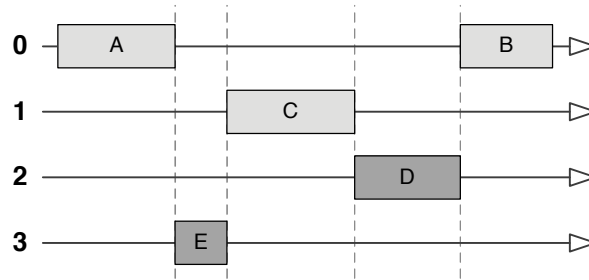


Figure 4.33. By wrapping each critical operation in a mutex lock-unlock pair, we force the operations to execute *sequentially*.

tions is transformed into an orderly sequential sequence of atomic operations. Making an operation atomic gives us no control over what the order will end up being; all we can say for certain is that the operations will be performed in *some* sequential order. This idea is illustrated in Figure 4.33.

#### 4.5.4.5 Data-Centric Consistency Models

The concepts of atomicity and the serialization of operations within a concurrent system are part of a larger topic known as *data-centric consistency models*. A consistency model is a *contract* between a data store, such as a shared data object in a concurrent system or a database in a distributed system, and a collection of threads that share that data store. It makes reasoning about the behavior of the data store easier—as long as the threads follow the rules of the contract, the programmer can be certain that the data store will behave in a consistent and predictable manner, and its data will not become corrupted.

A data store that provides a guarantee of atomicity can be said to be *linearizable*. The topic of data-centric consistency is a bit beyond our scope here, but you can read more about it online. Here are a few good places to start:

- Search for “consistency model” and “linearizability” on Wikipedia;
- <https://www.cse.buffalo.edu/~stevko/courses/cse486/spring13/lectures/26-consistency2.pdf>;
- <http://www.cs.cmu.edu/~srini/15-446/S09/lectures/10-consistency.pdf>.

## 4.6 Thread Synchronization Primitives

Every operating system that supports concurrency provides a suite of tools known as *thread synchronization primitives*. These tools provide two services to concurrent programmers:

1. The ability to share resources between threads by making critical operations *atomic*.
2. The ability to *synchronize* the operation of two or more threads:
  - a. by enabling a thread to *go to sleep* while it waits for a resource to become available or for one or more other threads to complete a task, and
  - b. by enabling a running thread to *notify* one or more sleeping threads by waking them up.

We should note here that while these thread synchronization primitives are robust and relatively easy to use, they are generally quite expensive. This is because these tools are provided by the kernel. Interacting with any of them therefore requires a *kernel call*, which involves a context switch into protected mode. Such context switches can cost upwards of 1000 clock cycles. Because of their high cost, some concurrent programmers prefer to implement their own atomicity and synchronization tools, or they turn to *lock-free programming* to improve the efficiency of their concurrent software. Nevertheless, a solid understanding of these synchronization primitives is an important part of any concurrent programmer’s toolkit.

### 4.6.1 Mutexes

A mutex is an operating system object that allows critical operations to be made atomic. A mutex can be in one of two states: *unlocked* or *locked*. (These two states are sometimes called *released* and *acquired*, or *signaled* and *nonsignaled*, respectively.)



The most important property of a mutex is that it guarantees that only one thread will ever be holding a lock on it at any given time. So, if we wrap all critical operations for a particular shared data object in a mutex lock, those operations become atomic relative to one another. In other words, the operations become *mutually exclusive*. This is where the name “mutex” comes from—it’s short for “mutual exclusion.”

A mutex is represented either by a regular C++ object or by a handle to an opaque kernel object. Its API is typically comprised of the following functions:

1. `create()` or `init()`. A function call or class constructor that creates the mutex.
2. `destroy()`. A function call or destructor that destroys the mutex.
3. `lock()` or `acquire()`. A blocking function that locks the mutex on behalf of the calling thread, but puts the thread to sleep (see Section 4.4.6.4) if the lock is currently held by another thread.
4. `try_lock()` or `try_acquire()`. A non-blocking function that attempts to lock the mutex, but returns immediately if the lock cannot be acquired.
5. `unlock()` or `release()`. A non-blocking function that releases the lock on the mutex. In most operating systems, only the thread that locked a mutex is permitted to unlock it.

When a mutex is locked by a thread in the system, we say it is in a *non-signaled* state. When the thread releases the lock, the mutex becomes *signaled*. If one or more other threads is asleep (blocked) waiting on the mutex, the act of signaling it causes the kernel to select one of these waiting threads and wake it up. In some operating systems, it’s possible for a thread to explicitly wait for a kernel object such as a mutex to become signaled. Under Windows, the `WaitForSingleObject()` and `WaitForMultipleObjects()` OS calls serve this purpose.

#### 4.6.1.1 POSIX

Now that we have an understanding of how mutexes work, let’s take a look at a few examples. The POSIX thread library exposes kernel mutex objects via a C-style functional interface. Here’s how we’d use it to turn our shared counter example from Section 4.5.3.2 into an atomic operation:

```
#include <pthread.h>

int g_count = 0;
pthread_mutex_t g_mutex;
```

```

inline void IncrementCount()
{
    pthread_mutex_lock(&g_mutex);
    ++g_count;
    pthread_mutex_unlock(&g_mutex);
}

```

Note that in the interest of clarity and brevity, we've omitted the code, normally executed by the main thread, that calls `pthread_mutex_init()` to initialize the mutex prior to spawning the threads that will use it, and that calls `pthread_mutex_destroy()` to destroy the mutex once all other threads have exited.

#### 4.6.1.2 C++ Standard Library

Starting with C++11, the C++ standard library exposes kernel mutexes via the class `std::mutex`. Here's how we'd use it to make the increment of a shared counter atomic:

```

#include <mutex>

int g_count = 0;
std::mutex g_mutex;

inline void IncrementCount()
{
    g_mutex.lock();
    ++g_count;
    g_mutex.unlock();
}

```

The constructor and destructor of the `std::mutex` class handles the initialization and destruction of the underlying kernel mutex object, making it a little bit easier to use than `pthread_mutex_t`.

#### 4.6.1.3 Windows

Under Windows, a mutex is represented by an opaque kernel object and referenced through a handle. A mutex is “locked” by waiting for it to become signaled, using the general-purpose `WaitForSingleObject()` function. Unlocking a mutex is accomplished by calling `ReleaseMutex()`. Rewriting our simple example using Windows mutexes, and again omitting the details of the creation and destruction of the mutex object, we arrive at the following code:

```
#include <windows.h>

int g_count = 0;
HANDLE g_hMutex;

inline void IncrementCount()
{
    if (WaitForSingleObject(g_hMutex, INFINITE)
        == WAIT_OBJECT_0)
    {
        ++g_count;
        ReleaseMutex(g_hMutex);
    }
    else
    {
        // learn to deal with failure...
    }
}
```

## 4.6.2 Critical Sections

In most operating systems, a mutex can be shared between processes. As such, it is a data structure that is managed internally by the kernel. This means that all operations performed on a mutex involve a kernel call, and hence a context switch into protected mode on the CPU. This makes mutexes relatively expensive, even when no other threads are contending for the lock.

Some operating systems provide less-expensive alternatives to a mutex. For example, Microsoft Windows provides a locking mechanism known as a *critical section*. The terminology and API look a bit different to that of a mutex, but a critical section under Windows is really just a low-cost mutex.

The API of a critical section looks like this:

1. `InitializeCriticalSection()`. Constructs a critical section object.
2. `DeleteCriticalSection()`. Destroys an initialized critical section object.
3. `EnterCriticalSection()`. A blocking function that locks a critical section on behalf of the calling thread, but busy-waits or puts the thread to sleep if the lock is currently held by another thread.
4. `TryEnterCriticalSection()`. A non-blocking function that attempts to lock a critical section, but returns immediately if the lock cannot be acquired.

5. `LeaveCriticalSection()`. A non-blocking function that releases the lock on a critical section object.

Here's how we'd implement an atomic increment using the Windows critical section API:

```
#include <windows.h>

int g_count = 0;
CRITICAL_SECTION g_critsec;

inline void IncrementCount()
{
    EnterCriticalSection(&g_critsec);
    ++g_count;
    LeaveCriticalSection(&g_critsec);
}
```

As before, we've omitted some details. The main thread normally initializes the critical section prior to spawning the threads that use it, and would of course clean it up once the threads have all exited.

How is the low cost of a critical section achieved? When a thread first attempts to enter (lock) a critical section that is already locked by another thread, an inexpensive *spin lock* is used to wait until the other thread has left (unlocked) that critical section. A spin lock does not require a context switch into the kernel, making it a few thousand clock cycles cheaper than a mutex. Only if the thread busy-waits for too long is the thread put to sleep, as it would be with a regular mutex. This less-expensive approach works because, unlike a mutex, a critical section cannot be shared across process boundaries. We'll discuss spin locks in more depth in Section 4.9.7.

Some other operating systems provide "cheap" mutex variants as well. For example, Linux supports a thing called a "futex" that acts somewhat like a critical section under Windows. Its use is beyond our scope here, but you can read more about futexes at <https://www.akkadia.org/drepper/futex.pdf>.

### 4.6.3 Condition Variables

In concurrent programming, we often need to send signals between threads in order to synchronize their activities. One example of this is the ubiquitous producer-consumer problem which we introduced in Section 4.4.8.1. In this problem, we have two threads: A *producer* thread calculates or otherwise generates some data, and that data is read and put to use by a *consumer* thread. Obviously the consumer thread cannot consume the data until the producer

has produced it. Hence the producer thread needs a way to notify the consumer that its data is ready for consumption.

We could consider using a global Boolean variable as a signalling mechanism. The following code snippet illustrates the idea, using POSIX threads. (Some details have been omitted for clarity.)

```
Queue          g_queue;
pthread_mutex_t g_mutex;
bool           g_ready = false;

void* ProducerThread(void*)
{
    // keep on producing forever...
    while (true)
    {
        pthread_mutex_lock(&g_mutex);

        // fill the queue with data
        ProduceDataInto(&g_queue);

        g_ready = true;
        pthread_mutex_unlock(&g_mutex);

        // yield the remainder of my timeslice
        // to give the consumer a chance to run
        pthread_yield();
    }
    return nullptr;
}

void* ConsumerThread(void*)
{
    // keep on consuming forever...
    while (true)
    {
        // wait for the data to be ready
        while (true)
        {
            // read the value into a local,
            // making sure to lock the mutex
            pthread_mutex_lock(&g_mutex);
            const bool ready = g_ready;
            pthread_mutex_unlock(&g_mutex);

            if (ready)
                break;
        }
    }
}
```

```

    }

    // consume the data
    pthread_mutex_lock(&g_mutex);
    ConsumeDataFrom(&g_queue);
    g_ready = false;
    pthread_mutex_unlock(&g_mutex);

    // yield the remainder of my timeslice
    // to give the producer a chance to run
    pthread_yield();
}
return nullptr;
}

```

Besides the fact that this example is somewhat contrived, there's one big problem with it: The consumer thread spins in a tight loop, polling the value of `g_ready`. As we discussed in Section 4.4.6.4, busy-waiting like this wastes valuable CPU cycles.

Ideally, we'd like a way to *block* the consumer thread (put it to sleep) while the producer does its work, and then wake it up when the data is ready to be consumed. This can be accomplished by making use of a new kind of kernel object called a *condition variable* (CV).

A condition variable isn't actually a variable that stores a condition. Rather, it's a queue of waiting (sleeping) threads, combined with a mechanism that allows a running thread to wake up the sleeping threads at a time of its choosing. (Perhaps "wait queue" would have been a better name for these things.) The sleep and wake operations are performed in an atomic way with the help of a mutex provided by the program, plus a little help from the kernel.

The API for a condition variable typically looks something like this:

1. `create()` or `init()`. A function call or class constructor that creates a condition variable.
2. `destroy()`. A function call or destructor that destroys a condition variable.
3. `wait()`. A blocking function that puts the calling thread to sleep.
4. `notify()`. A non-blocking function that wakes up any threads that are currently asleep waiting on the condition variable.

Let's rewrite our simple producer-consumer example using a CV:

```

Queue          g_queue;
pthread_mutex_t g_mutex;

```

```

bool          g_ready = false;
pthread_cond_t g_cv;

void* ProducerThreadCV(void*)
{
    // keep on producing forever...
    while (true)
    {
        pthread_mutex_lock(&g_mutex);

        // fill the queue with data
        ProduceDataInto(&g_queue);

        // notify and wake up the consumer thread
        g_ready = true;
        pthread_cond_signal(&g_cv);
        pthread_mutex_unlock(&g_mutex);
    }
    return nullptr;
}

void* ConsumerThreadCV(void*)
{
    // keep on consuming forever...
    while (true)
    {
        // wait for the data to be ready
        pthread_mutex_lock(&g_mutex);
        while (!g_ready)
        {
            // go to sleep until notified... the mutex
            // will be released for us by the kernel
            pthread_cond_wait(&g_cv, &g_mutex);

            // when it wakes up, the kernel makes sure
            // that this thread holds the mutex again
        }

        // consume the data
        ConsumeDataFrom(&g_queue);
        g_ready = false;
        pthread_mutex_unlock(&g_mutex);
    }
    return nullptr;
}

```

The consumer thread calls `pthread_cond_wait()` to go to sleep until

`g_ready` becomes true. The producer works for a while producing its data. When the data is ready, the producer sets the global `g_ready` flag to true, and then wakes up the sleeping consumer by calling `pthread_cond_signal()`. The consumer then consumes the data. In this example, the consumer and producer ping-pong back and forth like this indefinitely.

You probably noticed that the consumer thread locks its mutex before entering the while loop that checks the `g_ready` flag. When it waits on the condition variable, it apparently goes to sleep *while holding the mutex lock*! Normally this would be a big no-no: If a thread goes to sleep while holding a lock, it will almost certainly lead to a *deadlock* situation (see Section 4.7.1). However, this is not a problem when using a condition variable. That's because the kernel actually does a little slight of hand, *unlocking the mutex* after the thread has been safely put to bed. Later, when the sleeping thread is woken back up, the kernel does some more slight of hand to ensure that the lock will once again be held by the freshly awoken thread.

You may have noticed another oddity: The consumer thread still uses a while loop to check the value of `g_ready`, despite also using a condition variable to wait for the flag to become true. The reason that this loop is necessary is that threads can sometimes be awoken spuriously by the kernel. As a result, when the call to `pthread_cond_wait()` returns, the value of `g_ready` might *not* actually be true yet. So we must keep polling in a loop until the condition really is true.

#### 4.6.4 Semaphores

Just as a mutex acts like an atomic Boolean flag, a *semaphore* acts like an atomic counter whose value is never allowed to drop below zero. We can think of a semaphore as a special kind of mutex that allows *more than one thread* to acquire it simultaneously.

A semaphore can be used to permit a group of threads to share a limited set of resources. For example, let's suppose that we're implementing a rendering system that allows text and 2D images to be rendered to offscreen buffers, for the purposes of drawing the game's heads-up display (HUD) and in-game menus. Due to memory constraints, let's further assume that we can only afford to allocate four of these buffers. A semaphore can be used to ensure that no more than four threads are permitted to render into these buffers at any given moment.

The API of a semaphore is typically comprised of the following functions:

1. `init()`. Initializes a semaphore object, and sets its counter to a specified



initial value.

2. `destroy()`. Destroys a semaphore object.
3. `take()` or `wait()`. If the counter value encapsulated by a given semaphore is greater than zero, this function decrements the counter and returns immediately. If its counter value is currently zero, this function blocks (puts the thread to sleep) until the semaphore's counter rises above zero again.
4. `give()`, `post()` or `signal()`. Increments the encapsulated counter value by one, thereby opening up a "slot" for another thread to `take()` the semaphore. If a thread is currently asleep waiting on the semaphore when `give()` is called, that thread will wake up from its call to `take()` or `wait()`.<sup>7</sup>

So, to implement a resource pool that can be accessed by up to  $N$  threads at a time, we would simply create a semaphore and initialize its counter to  $N$ . A thread gains access to the resource pool by calling `take()`, and releases its hold on the resource pool when it is done by calling `give()`.

We say that a semaphore is *signaled* whenever its count is greater than zero, and it is *nonsignaled* when its counter is equal to zero. This is why the functions that take and give a semaphore are named `wait()` and `signal()`, respectively, in some APIs: If the semaphore isn't signaled when a thread calls this function, the thread will *wait* for the semaphore to become signaled.

#### 4.6.4.1 Mutex versus Binary Semaphore

A semaphore whose initial value is set to 1 is called a *binary semaphore*. One might think that a binary semaphore is identical to a mutex. Certainly both objects permit only one thread to acquire it at a time. However, these two synchronization objects are not equivalent, and are typically used for quite different purposes.

The key difference between a mutex and a binary semaphore is that a mutex can only be unlocked by the thread that locked it. A semaphore's counter, on the other hand, can be incremented by one thread and later decremented by another thread. This implies that a binary semaphore can be "unlocked" by a different thread than the one that "locked" it. Or really, we should say that a binary semaphore can be *given* by a different thread than the one that *takes* it.

This seemingly subtle difference between mutexes and binary semaphores leads to very different use cases for these two kinds of synchronization object.

<sup>7</sup>When you read about semaphores, you may discover some authors using the function names `p()` and `v()` instead of `wait()` and `signal()`. These letters come from the Dutch names for these two operations.

A mutex is used to make an operation *atomic*. But a binary semaphore is typically used to send a *signal* from one thread to another.

Consider again our producer-consumer example, in which the producer needs to notify the consumer when the data it produces is ready for consumption. This notification mechanism can be implemented using two binary semaphores, one that allows the producer to wake the consumer, and one that allows the consumer to wake the producer. We can think of these semaphores as representing the number of elements that are used and free within a buffer that's shared between the two threads, although in this simple example the buffer can only hold a single item. As such, we'll call the semaphores `g_semUsed` and `g_semFree`, respectively. Here's what the code would look like, using POSIX semaphores:

```
Queue g_queue;
sem_t g_semUsed; // initialized to 0
sem_t g_semFree; // initialized to 1

void* ProducerThreadSem(void*)
{
    // keep on producing forever...
    while (true)
    {
        // produce an item (can be done non-
        // atomically because it's local data)
        Item item = ProduceItem();

        // decrement the free count
        // (wait until there's room)
        sem_wait(&g_semFree);

        AddItemToQueue(&g_queue, item);

        // increment the used count
        // (notify consumer that there's data)
        sem_post(&g_semUsed);
    }
    return nullptr;
}

void* ConsumerThreadSem(void*)
{
    // keep on consuming forever...
    while (true)
    {
        // decrement the used count
```

```

        // (wait for the data to be ready)
        sem_wait(&g_semUsed);

        Item item = RemoveItemFromQueue(&g_queue);

        // increment the free count
        // (notify producer that there's room)
        sem_post(&g_semFree);

        // consume the item (can be done non-
        // atomically because it's local data)
        ConsumeItem(item);
    }
    return nullptr;
}

```

#### 4.6.4.2 Implementing a Semaphore

It turns out that one can implement a semaphore in terms of a mutex, a condition variable and an integer. In that sense, a semaphore is a “higher-level” construct than either a mutex or a condition variable. Here’s what the implementation looks like:

```

class Semaphore
{
private:
    int m_count;
    pthread_mutex_t m_mutex;
    pthread_cond_t m_cv;

public:
    explicit Semaphore(int initialCount)
    {
        m_count = initialCount;
        pthread_mutex_init(&m_mutex, nullptr);
        pthread_cond_init(&m_cv, nullptr);
    }

    void Take()
    {
        pthread_mutex_lock(&m_mutex);
        // put the thread to sleep as long as
        // the count is zero
        while (m_count == 0)
            pthread_cond_wait(&m_cv, &m_mutex);
    }
}

```

```

        --m_count;
        pthread_mutex_unlock(&m_mutex);
    }

    void Give()
    {
        pthread_mutex_lock(&m_mutex);
        ++m_count;
        // if the count was zero before the
        // increment, wake up a waiting thread
        if (m_count == 1)
            pthread_cond_signal(&m_cv);
        pthread_mutex_unlock(&m_mutex);
    }

    // aliases for other commonly-used function names
    void Wait() { Take(); }
    void Post() { Give(); }
    void Signal() { Give(); }
    void Down() { Take(); }
    void Up() { Give(); }
    void P() { Take(); } // Dutch "proberen" = "test"
    void V() { Give(); } // Dutch "verhogen" =
                        // "increment"
};

```

#### 4.6.5 Windows Events

Windows provides a mechanism called an *event object* that is similar in function to a condition variable, but much simpler to use. Once an event object has been created, a thread can go to sleep by calling `WaitForSingleObject()`, and that thread can be awoken by another thread by calling `SetEvent()`. Rewriting our producer-consumer example using event objects yields the following very simple implementation:

```

#include <windows.h>

Queue g_queue;
Handle g_hUsed; // initialized to false (nonsignaled)
Handle g_hFree; // initialized to true (signaled)

void* ProducerThreadEv(void*)
{
    // keep on producing forever...
    while (true)
    {
        // produce an item (can be done non-

```

```

        // atomically because it's local data)
        Item item = ProduceItem();

        // wait until there's room
        WaitForSingleObject(&g_hFree);

        AddItemToQueue(&g_queue, item);

        // notify consumer that there's data
        SetEvent(&g_hUsed);
    }
    return nullptr;
}

void* ConsumerThreadEv(void*)
{
    // keep on consuming forever...
    while (true)
    {
        // wait for the data to be ready
        WaitForSingleObject(&g_hUsed);

        Item item = RemoveItemFromQueue(&g_queue);

        // notify producer that there's room
        SetEvent(&g_hFree);

        // consume the item (can be done non-
        // atomically because it's local data)
        ConsumeItem(item);
    }
    return nullptr;
}

void MainThread()
{
    // create event in the nonsignalled state
    g_hUsed = CreateEvent(nullptr, false,
                          false, nullptr);
    g_hFree = CreateEvent(nullptr, false,
                          true, nullptr);

    // spawn our threads
    CreateThread(nullptr, 0x2000, ConsumerThreadEv,
                0, 0, nullptr);
    CreateThread(nullptr, 0x2000, ProducerThreadEv,
                0, 0, nullptr);
}

```

```
    // ...  
}
```

## 4.7 Problems with Lock-Based Concurrency

In Section 4.5.3.2, we learned that *data races* can lead to incorrect program behavior in a concurrent system. We saw that the solution to this problem is to make operations on *shared* data objects *atomic*. One way to achieve atomicity is to wrap these operations in *locks*, which are often implemented using operating-system provided thread synchronization primitives such as mutexes.

However, atomicity is only part of the concurrency story. There are other problems that can plague a concurrent system, even when all shared-data operations have been carefully protected by locks. In the following sections, we'll briefly explore the most common of these problems.

### 4.7.1 Deadlock

*Deadlock* is a situation in which no thread in the system can make progress, resulting in a hang. When a deadlock occurs, all threads are in their Blocked states, waiting on some resource to become available. But because no threads are Runnable, none of those resources can ever become available, so the entire program hangs.

For a deadlock to occur, we need at least two threads and two resources. For example, Thread 1 holds Resource A but is waiting for Resource B; and at the same time, Thread 2 holds Resource B but is waiting for Resource A. This situation is illustrated by the following code snippet:

```
void Thread1()  
{  
    g_mutexA.lock(); // holds lock for Resource A  
    g_mutexB.lock(); // sleeps waiting for Resource B  
    // ...  
}  
  
void Thread2()  
{  
    g_mutexB.lock(); // holds lock for Resource B  
    g_mutexA.lock(); // sleeps waiting for Resource A  
    // ...  
}
```